

Architecture over Vibes

Agentic Engineering for Security Practitioners

Ethan Seow · Practical Cyber × C4AIL

DEF CON Asia 2026 — AI & AI Security Kampung

Companion paper: [The Architecture of AI-Assisted Development — From Vibe Coding to Sovereign Command](#) · C4AIL, March 2026

The frame

The most dangerous thing in 2026 is not an AI that writes bad code. It's a human who can't tell when AI wrote bad code.

- My concern isn't capability — Claude Code, Codex, Gemini CLI are genuinely great.
- My concern is engineers who never learned to **reject** a suggestion.
- **Architecture is the discipline of rejection. Syntax is the easy part.**

This is the **Translator capability** in code form: I'm not faster than the AI at typing. I'm faster than the AI at knowing what **should** be typed — and at **making it run the actual code** so the answer is verified, not vibed.

The five ages of AI-assisted code

A maturity progression, not a timeline. The industry walked through four wrong defaults to arrive at the fifth.

Age	Year	Default move	Where it broke
1. Prompt engineering	2023-24	Craft a smarter prompt	Better prompts, same unreliable output
2. Vibe coding	Feb 2025 (Karpathy)	"Forget the code even exists"	Three-month wall — nobody can debug it
3. The 70% Problem	Late 2024 (Osmani)	AI gets 70% done fast	The last 30% is human accountability labour
4. Context engineering	Jun 2025 (Lutke)	Design the environment, not the prompt	Still measures tooling, not human judgment
5. Agentic engineering	Feb 2026 (Karpathy: vibe coding "passé")	Human owns architecture & verification; AI implements	This is where we are

Spec-Driven Development is its formal methodology. AWS Kiro, GitHub Spec Kit, SPARC, Intent, OpenSpec — all the same shape: **spec is the source of truth, code is the build output.**

The evidence against vibes

Four numbers from independent research. Internalise these before you accept your next AI suggestion.

–19%

Experienced devs **slower** on their own codebases with AI assistance — while believing they were 20% faster (METR RCT, 2025)

–17%

Skill assessment drop for devs using AI coding assistants. Largest gaps in **comprehension and debugging** — the exact skills needed to verify AI output (Anthropic, 2025)

45%

of AI-generated code contains security vulnerabilities without structural verification (Veracode, 2025)

+98%

PRs merged · review time grew only +91% — senior devs reviewing 6.5% more code while their own output declined (Faros AI, 10K+ devs)

The Eloquence Trap applied to productivity: the experience of speed is not the reality of speed. The tool degrades the skill the tool requires.

Verification debt

Werner Vogels (AWS CTO) named the accumulating cost: **verification debt**.

"When you write code yourself, comprehension comes with the act of creation — but when the machine writes it, you must rebuild that comprehension during review. Every line of AI-generated code that ships without genuine understanding is debt that will come due."

The honest counterweight: human-written code didn't always come with built-in comprehension either. Cargo-culted Stack Overflow has existed for two decades.

The real loss is specific: **the cognitive trace** — the memory of why each decision was made — that made a skilled engineer's code maintainable. AI removes the act of creation, and with it the trace.

The fix is not to reject AI. It's to encode the trace into something machine-readable: instruction files, specs, gotchas, verification chains. Architecture beats audit.

What context engineering actually is

Lutke (Shopify CEO, Jun 2025): "providing all the context needed for a task to be plausibly solvable by an AI system." Practitioners refined this into four components.

Component	What it is	Example
Instructions	Rules, constraints, anti-patterns. Architecture, not suggestions.	"Never create docs unless asked." "Mandatory plan before edits."
Knowledge	Domain context. Past failures. Edge cases. The ORM behaviour that surprises everyone.	"kerykeion v5 <code>mean_node</code> returns None — use <code>true_node</code> ."
Tools	Capabilities the AI can invoke — files, APIs, deploys.	MCP servers, CLI tools, custom validators
State	What's been done, what's in progress, what failed.	Task system, session memory, audit log

Prompt engineering asked: "How do I phrase this so the AI gives me what I want?" Context engineering asks: "What architecture do I need so the AI's **default** output meets my standards?"

The five knowledge layers

A senior engineer's judgment is different from a junior's prompt because humans process five layers simultaneously. AI natively processes the bottom two.

Layer	What it is	Where it lives in your repo
Experiential	Body-level pattern recognition from years of failures	Gotchas — "this broke us in production"
Contextual	This codebase's patterns, this domain's edge cases	Project-specific instructions
Institutional	How the team works, what the culture permits	Coding conventions, review norms
Deductive	Formal reasoning, frameworks, rubrics	Sizing heuristics, validators
Syntactic	Format, naming, surface language	The layer AI already handles

Vibe coding fails for the same reason factory education fails: it operates on syntactic + deductive only. Context engineering is the discipline of encoding the missing three layers into the AI's environment.

The spec is the leverage

Your spec sheet is more powerful than the AI itself. Model choice is a knob. Spec quality is the multiplier.

- **Sean Grove (OpenAI), The New Code:** "the specification is the new source code." The model is a compiler; the spec is what you actually author.
- **Karpathy (2026):** the most valuable artefact in an AI-native codebase is the written specification — the model is replaceable, the spec compounds.
- **Kiro · GitHub spec-kit · Claude Code skills:** spec → plan → tasks is now the native loop. The agent doesn't free-associate; it executes against a written contract.
- **The mechanic:** a sharp `.md` spec lets Opus produce production-grade output without burning toward a million-token context. The spec compresses intent so the model doesn't re-derive it every turn.
- **The Translator move:** leaders don't write code, they write specs. That's how you commission AI work without becoming a coder — and it's how you stay accountable for what ships.

Practical rule: if your spec fits in a focused `.md` file with examples and rejection criteria, you can run a smaller context window with higher quality. The discipline is upstream of the model.

Where you actually run this

Browser chat is a toy. Real agentic engineering lives in Linux — because that's where your code will ship.

- **Secure VPS + container** is the substrate. Auto / bypass-permissions mode is reckless on your laptop and reasonable inside an isolated Docker container with a bounded blast radius.
- **Why containers:** reproducible, network-bounded, snapshot-able, and the **same artefact** that goes to production. You stop testing in one environment and deploying to another.
- **Spec lives in `.md` . Agent runs in a container. Deployment is the artefact you already tested.** That's the whole loop.
- **Roost** — my open-source AI nest, the answer to OpenClaw. Self-hosted on a VPS, persistent memory, MCP tools, Guardian pre-flight checks on every tool call, autonomy levels (supervised / assisted / autonomous), cost limits, checkpoints + rollback.
- github.com/crazyguy106/roost · one-command install on a fresh Ubuntu host · MIT-spirit · clone, fork, run.

The deployment story is the security story. An agent with bypass-permissions on your laptop is a breach waiting to happen. The same agent, same permissions, inside a container on a VPS, is just an engineer.

What you're about to see

A real security tool, built in front of you, in the next eighteen minutes.

- Audience picks the tool. Nothing pre-written.
- Same Claude Code I use on a Tuesday at my desk — running against a CLAUDE.md I have spent six months refining.
- **At least once, I will reject something Claude writes**, out loud, with reasoning.
- **At least once, I will make Claude run the actual code** — because **running it** is how the answer stops being a vibe.

That rejection and that test run are the whole point. The code that ships is what's left after I say no. The code that's **verified** is what's left after I make it execute.

Fallback prompts on the card: S3 → mask SG NRICs → write redacted manifest · tail Suricata alerts → de-dupe + enrich → Slack · classify failed SSH on a honeypot — bot vs targeted human

Three things I'm watching for

1 — Did it make a security decision I should be making?

"Should this be encrypted at rest." "Who can call this endpoint." "What's the failure mode." Silent decisions there → I roll back.

2 — Did it hide a side effect?

AI loves to `os.remove`, `git push`, `subprocess.run` in the middle of a function called `parse_thing`. **Function name should equal the function's blast radius.**

3 — Did it pretend to handle an error it cannot handle?

`try / except / pass` blocks are the new SQL injection. They turn a loud failure into a silent corruption.

Anything else, I let Claude type. Then I make it run pytest. **Running the code is the cheapest form of accountability labour.**

The build cadence

Min	Phase	What I do	What you watch for
0-1	Spec it	Write 5-line <code>SPEC.md</code> in plain English	Can I state the tool in 5 lines? If not, I don't understand it yet
1-3	Skeleton it	Ask Claude for file layout. Reject one suggestion.	"No <code>utils.py</code> — that's where bugs go to hide"
3-6	Tests first	3 deterministic tests. Claude can suggest; I gate.	Tests are the spec, compiled
6-11	Implement + run	Claude writes. I read. Then I make it run pytest after every change.	The planned moment — see next slide
11-13	Verify	Tests red → green. Green-on-first → suspicious — add edge case.	Easy green = missing test

If **any** phase finishes faster than expected, I add tests or run a hostile input. **Time saved is risk surfaced.**

The planned rejection

~minute 10. Regardless of what Claude writes, I'll find an excuse.

```
# What Claude wrote:
try:
    result = boto3.client('s3').get_object(...)
except Exception:
    pass

# What I replace it with:
try:
    result = boto3.client('s3').get_object(...)
except (BotoCoreError, ClientError) as e:
    logger.error("S3 fetch failed: %s", e)
    raise
```

Three differences:

1. I named the exceptions I expected.
2. I logged it.
3. I re-raised so the caller can decide.

Then I run pytest. Because **saying** the fix is right is a vibe. **Running it** is verification.

Why that rejection matters — the labour split

Claude does the intellectual labour

- Figured out which AWS calls to make
- Wired up the SDK
- Generated the boilerplate
- Suggested the test structure

I do the accountability labour

- Decide what happens when the call fails
- Decide what gets logged
- **Make it run the actual code**
- Defend it in the postmortem

`except Exception: pass` is the most dangerous line in modern software. It says "if anything goes wrong I don't care, return success anyway." That decision belongs to the person whose name is on the on-call rotation — not to a probability machine.

Decision Survivability: can you defend this after it breaks?

Read > write > vibe — the 80/15/5

What you'll watch most of these eighteen minutes is me **reading** and **running**, not typing.

~80% reading — spec, tests, generated code, diff before commit, test output

20% writing

- Read the spec.
- Read the tests.
- Read every line Claude generated.
- **Run pytest.** Read the actual output. Not "Claude said it passes" — **run it.**
- Read the diff before I commit.

Self-reported metric on my own context-engineering rig: 50-80% first-time-right on bulk deliverable generation. Took six months and roughly 4,300 lines of structured CLAUDE.md across 29 files to get there. **The discipline, not the model, is the compounding asset.**

This isn't just my framing

Liu, Zhao, Shang, Shen — Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems. arXiv 2604.14228, 14 Apr 2026.

Independent analysis of Claude Code's TypeScript source:

"A simple while-loop that calls the model, runs tools, and repeats... **most of the code, however, lives in the systems around this loop.**"

The five values they extracted from the source:

1. **Human decision authority** ← the rejection moment, peer-reviewed
2. Safety / security
3. Reliable execution
4. Capability amplification
5. Contextual adaptability

The tool I'm using right now is itself **98% deterministic harness, 2% model call**. Permission system, context compaction, tool dispatch, session storage — all code. The intelligence is the **edge**. Same architecture I'm asking you to build into your own tools.

The one habit

If you walk away with one practice from this session:

Before you accept any AI-generated code,

say out loud what you would have written instead — then run the code to check.

If you can do neither, you don't understand it well enough to ship it.

That's the bar. That's the 98/2. That's agentic engineering in your IDE.

AI's value comes from your data — the spec, the tests, the constraints. Its security comes from your framework — your refusal, your logging, your error model, your **actual test runs**. **Vibe coding doesn't change the fundamentals. It just makes them visible faster.**

Bridge to Mission 3

Want to feel this in your hands today? **Mission 3 — TDD Speedrun** is the one.

- Same loop: human writes the tests, AI writes the implementation, **human runs the tests**.
- Thirty minutes. Ninety per cent coverage.
- The award goes to the team whose **rejected suggestions** were the most interesting.
- Keep a one-line note every time you say no to Claude — and every time you caught it by **running** the code.
- **That note is the deliverable.** It's the rubric tiebreaker.

Close

"I architect. AI types. I make it run.

If you can't tell me which line you'd reject — or which test you'd run — you're not vibe coding. You're being driven."



defcon.practical-cyber.com — [slides](#) · [talks](#) · [missions](#) · [starter kits](#)

Ethan Seow · ethan@practical-cyber.com